

Lab 6: Queues and Deques

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Queue ADT, circular array, double-ended queue

Objectives

- Define the Queue ADT (FIFO) and the Deque ADT (double-ended).
- Implement a queue backed by a circular array using modular arithmetic.
- Implement a Deque using a doubly linked list.
- Apply queues to a round-robin task scheduler simulation.

Quick Reference

Queue interface.

```
void enqueue(E e);    // add to back
E dequeue();          // remove from front
E first();             // peek front, no removal
int size(); boolean isEmpty();
```

Circular array. Maintain front and size. Back index: $(\text{front} + \text{size}) \% \text{capacity}$.

```
// enqueue:
data[(front + size) % capacity] = e; size++;
// dequeue:
E val = data[front];
front = (front + 1) % capacity; size--;
return val;
```

Deque API. `addFirst`, `addLast`, `removeFirst`, `removeLast`, `peekFirst`, `peekLast` — all $O(1)$.

Quick Experiments (Try It)

Try It 1: Wrap-around with a small capacity

Create an `ArrayQueue<Integer>` of capacity 4. Enqueue 1, 2, 3, 4. Dequeue twice. Enqueue 5, 6. Print front and the logical contents. Confirm front has wrapped around.

Try It 2: Queue vs. Stack order

Push numbers 1–5 onto a stack, then pop them into a queue. Dequeue and print. What order do you see? What did the stack-to-queue transfer accomplish?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Array-Based Queue (40 min)

Implement `ArrayQueue<E>` backed by a fixed-capacity `Object[]`.

- Fields: `data[]`, `front`, `size`, `capacity`.
- `enqueue` throws `IllegalStateException` when full.
- `dequeue` and `first` throw `NoSuchElementException` when empty.
- `toString()` shows front-to-back, e.g. `front->[1, 2, 3]<-back`.

Test with wrap-around: capacity 5, enqueue 5, dequeue 2, enqueue 2 more.

In-Lab Exercise 2: Linked-List Deque (30 min)

Using `DoublyLinkedList<E>` from Lab 4, wrap it in `LinkedListDeque<E>` with the full Deque API. All operations must be $O(1)$. Write `boolean isPalindrome(String s)` using the deque. Test on "level", "world", "noon".

In-Lab Exercise 3: Round-Robin Scheduler (30 min)

Simulate a round-robin CPU scheduler:

1. Task has fields `name` (String) and `burstTime` (int ms).
2. Read at least 4 tasks from the user.
3. Time quantum = 3 ms. Each round: dequeue, subtract quantum (or finish if $\leq$ quantum), print a log line, re-enqueue if remaining time > 0 .
4. Continue until the queue is empty.

Example log: [t=6] Running TaskA, remaining=4ms.

AI Collaboration

Suggested prompts:

- “Why does a circular array avoid wasting space that a naive queue wastes?”
- “What is the difference between Deque and Queue in Java’s standard library?”

Verify: Ask the AI to draw the circular array state after the wrap-around sequence in Try It

1. Compare with your own diagram.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Circular array trace

Trace on an `ArrayQueue` of capacity 4. After each operation record `front`, `size`, and the array (– for empty slots).

```
enqueue(A); enqueue(B); enqueue(C); dequeue();
enqueue(D); enqueue(E); dequeue(); dequeue();
```

Write Code on Paper 1: Reverse a queue using a stack

Write `void reverseQueue(Queue<Integer> q)` that reverses all elements in `q` using only one auxiliary `Stack<Integer>`. No other data structures allowed.

Draw on Paper 1: Round-robin Gantt chart

Draw a Gantt chart for these tasks with quantum = 2 ms:

Task	Burst	Arrival
P1	6 ms	0
P2	4 ms	0
P3	2 ms	0
P4	5 ms	0

Submission. Save files as `lab6_ex*.java` and submit on the course site.